

GALILEO Graph Databases Group
**Implementation of GALOIS Features – Second
Check**

(Queries 18–30, Replication of Table 3 – Third
Column)

Francesco Pivotto 2158296
Giorgia Amato 2159999
Alessio Demo 2142885

December 3, 2025

Contents

1	Introduction & Goals of GALOIS Implementation	2
2	Iterative Baseline Extension	2
2.1	Motivation and Objective	2
2.2	Design Principles and Architectural Integration	3
2.3	Iterative Execution Workflow	3
2.4	Baseline-Specific Considerations	3
2.5	Outcome and Observations	4
3	Evaluation of the Iterative Baselines	4
3.1	Iterative Execution Results	4
3.2	Comparison with Non-Iterative Baselines	5
4	GALOIS system architecture	5
4.1	Key Retrieval Phase (First Stage)	5
4.2	Table-Scan Execution Phase (Second Stage)	5
4.3	Table vs Key-Scan in GaloisWO	6
5	Logical Optimization (Query PLanning)	6
5.1	Predicate Push-Down Strategy	7
5.2	Join Handling	7
6	Physical Optimization (Execution Strategies)	7
6.1	Table-Scan Operator	7
6.2	Key-Scan Operator	8
6.3	Adaptive Strategy Selection	8

7	Execution Cycle & Memory Handling	8
7.1	Stateful Context Management	8
7.2	The Iterative execution loop	8
7.3	Termination Logic and Convergence	8
8	Post-Processing	8
8.1	Data Aggregation and Virtualization	9
8.2	Heuristic Type Inference and Sanitization	9
8.3	Deterministic Query Execution	9
9	Numerical Results	9
9.1	GALOIS WO Results comparison	10
9.2	GALOIS S Results comparison	10
9.3	GALOIS A Results comparison	10
9.4	GALOIS F Results comparison	10
10	Implementative challenges encountered	11
11	Contributions	11

1 Introduction & Goals of GALOIS Implementation

Large Language Models (LLMs) can answer declarative queries expressed in Natural Language (NL) or SQL, but their one-shot responses frequently fail to recover complete and factually consistent tuples when queries involve filtering, joins, or aggregations. As highlighted in the GALOIS paper, direct prompting exhibits limited recall and incomplete enumeration, making LLMs unreliable as standalone query processors.

The GALOIS framework mitigates these limitations by treating the LLM as a latent knowledge store and decomposing execution into logical and physical operators, in analogy with traditional DBMSs. Rather than issuing a single prompt, GALOIS executes structured scan and filtering strategies (Table Scan, Key Scan) together with multiple logical planning variants (push-all, push-selective, push-wo, and the optimized GALOIS-F). This operator-based design improves tuple completeness and stabilizes execution across heterogeneous query types.

2 Iterative Baseline Extension

2.1 Motivation and Objective

Building on the unified execution pipeline introduced in previous phases, this task extends both the SQL and NL baselines with a controlled iterative retrieval mechanism. The goal is to improve result completeness by allowing the LLM to generate multiple partial result sets across successive calls, while preserving full compatibility with the project’s standardized JSON output schema.

This mechanism addresses cases where a single LLM invocation fails to recover the entire expected answer set due to truncation, prompt limitations, or generation variability. Iteration provides a systematic way to accumulate non-overlapping tuples across rounds.

2.2 Design Principles and Architectural Integration

The iterative enhancement was introduced without modifying the core architecture. A configurable parameter `max_iter` (defined in `config.yaml`) controls the behaviour and ensures consistent execution across providers.

The integration adheres to the following principles:

- **Provider-Agnostic Execution.** Iteration is implemented at the baseline level (SQL and NL), leaving LLM instantiation and LCEL chain construction unchanged. This guarantees full compatibility with any provider supported through the Adapter and Factory patterns.
- **Minimal Invasiveness.** No modifications were made to the LCEL chain, output parser, or prompt template. Iteration is strictly an additional layer wrapped around the existing call-parse-store cycle.
- **Deduplicated Accumulation.** Each iteration contributes new tuples based on strict key-wise comparison. A local accumulator collects all rows returned in previous rounds, preventing redundancy and guaranteeing stable output.
- **Early Stopping Criterion.** To avoid unnecessary LLM calls, the system terminates early if an iteration yields zero new rows. This rule applies identically to SQL and NL baselines.

2.3 Iterative Execution Workflow

For each query (SQL baseline) or natural language prompt (NL baseline), the extended workflow proceeds as follows:

1. **Initialization.** Create an accumulator for unique tuples and load `max_iter`.
2. **Repeated Invocation.** At each iteration, the baseline executes:
 - (a) construction of the LCEL chain with the existing prompt template,
 - (b) LLM invocation using the configured provider,
 - (c) parse the response via the Pydantic output parser.
3. **Deduplication and Union.** Compare new tuples to the accumulator using canonical JSON formatting; retain only unseen tuples.
4. **Stopping Condition.** Terminate when no new tuples are found.
5. **Persistence.** After early stopping or reaching `max_iter`, serialize:
 - the union of all retrieved tuples,
 - total elapsed time,
 - aggregated token usage.

2.4 Baseline-Specific Considerations

SQL Baseline. Iteration compensates for incomplete tuple enumeration in a single pass. Because SQL inputs are fully specified, repeated calls primarily improve recall.

NL Baseline. Iteration mitigates incomplete enumerations and semantic drift in free-form queries. Early stopping is essential to prevent hallucinated or reformulated rows once the LLM stabilizes on a fixed subset.

2.5 Outcome and Observations

Controlled iteration increases tuple coverage while remaining fully aligned with the evaluation framework. The design preserves modularity, provider-agnosticism, and the standardized JSON format established in previous phases.

The mechanism also maintains conceptual continuity with iterative operators in the GALOIS framework (e.g., Key-Scan), providing a foundation for more advanced retrieval strategies in future work.

3 Evaluation of the Iterative Baselines

We evaluated the iterative SQL and NL baselines on the full benchmark suite (FLIGHT-2, FLIGHT-4, GEO, MOVIES, PRESIDENTS, WORLD).

3.1 Iterative Execution Results

First of all for what concerns the `llama-3.3-70b-instruct` model, the following table highlights the differences in terms of metrics between results presented in the original GALOIS paper and our implementation with iterative extension:

Metric	PAPER	Ours	$\Delta(\text{PAPER} - \text{Ours})$
F1-Cell	0.431	0.403	0.028
Cardinality	0.659	0.465	0.194
Tuple Constr.	0.351	0.197	0.154
AVG-Score	0.481	0.355	0.126
#Tokens (M)	0.330	0.008	0.322
Avg Time	61.400	5.263	56.137

Table 1: sql_baseline

Metric	PAPER	Ours	$\Delta(\text{PAPER} - \text{Ours})$
F1-Cell	0.237	0.271	-0.034
Cardinality	0.462	0.500	-0.038
Tuple Constr.	0.065	0.091	-0.026
AVG-Score	0.254	0.287	-0.033
#Tokens (M)	0.830	0.005	0.825
Avg Time	120.000	2.206	117.794

Table 2: nl_baseline

Maintaining consistency with the previous report, the following table show the results obtained with the iterative extension for both baselines using `gemini-2.5-flash` models:

The iterative execution impacts the behaviour of both baselines in distinct ways. In the SQL setting, the model benefits from task repetition, often recovering additional valid

Metric	NL	SQL
F1-CELL	0.425	0.456
CARDINALITY	0.696	0.607
TUPLE-CONSTR.	0.262	0.245
#TOKENS (M)	0.028	0.067
AVG-TIME (s)	5.035	9.681
AVG-SCORE	0.461	0.436

Table 3: Iterative Results for `gemini-2.5-flash`.

tuples that were missed in the first pass. This results in higher tuple-level completeness and, in many cases, improvements in the F1-CELL and CARDINALITY metrics. For NL queries, which are inherently more open-ended, the iterative mechanism still enhances tuple coverage, though with a less uniform effect across datasets.

3.2 Comparison with Non-Iterative Baselines

To contextualise the effect of iteration, we compare the results above with the single-pass baselines. Overall, the iterative variant provides measurable gains in tuple completeness and global AVG-SCORE, especially in datasets where the model tended to omit large fractions of the answer in a single call. As expected, the iterative mechanism also increases the total number of generated tokens and the average execution time. However, these costs remain moderate and proportional to the configured value of `max_iter`.

In conclusion, iterative execution improves robustness and recall, providing a more stable foundation for evaluating LLM behaviour under both SQL and NL interfaces.

4 GALOIS system architecture

This section describes the implementation of **GALOISWO** (Galois Without Optimizations), the baseline version of the Galois framework implemented for this checkpoint. Unlike the full Galois system, GaloisWO does not apply any logical or physical optimizations such as confidence-based pushdown, selective operator choice, metadata reasoning, or cost-quality trade-off. Instead, it adopts a simple two-stage execution pipeline:

1. **Key Extraction:** retrieve all possible primary keys of the target table using iterative prompting.
2. **Tuple Completion:** for each retrieved key, prompt the LLM to obtain remaining attribute values (one tuple at a time).

4.1 Key Retrieval Phase (First Stage)

4.2 Table-Scan Execution Phase (Second Stage)

In the Galois paper, two physical scan operators are introduced: *Table-Scan* and *Key-Scan*. While the full Galois system dynamically selects between them according to confidence and metadata, GaloisWO uses only **Table-Scan** in a fully deterministic manner, with no optimization or pushdown logic.

This behavior is clearly reflected in our implementation, specifically in the `table_scan()` method of the `GaloisExecutor` class. The execution shows the following key characteristics:

- **No condition pushdown:** The executor uses the original SQL query, but does not apply any optimization selection. The WHERE clause is either used as-is or fully omitted if not available. There is no selection of pushable conditions based on metadata or confidence values.
- **Schema-aware attribute discovery:** The executor retrieves full table attribute lists using the `GaloisWOSchemaManager`, which is a thin wrapper around the standard schema manager but isolates the `GaloisWO` namespace.
- **First Prompt and Iterative Prompt Generation:** The system constructs the first and iterative human prompts using: `build_table_scan_first_prompt()` and `build_table_scan_iter_prompt()` from `galois_prompts.py`.

These prompts enforce:

- valid JSON-only output,
 - table-aware attribute listing,
 - a maximum of 20 tuples per iteration,
 - and no natural language explanations.
- **Iterative Expansion:** The executor issues multiple LLM calls (up to `max_iter`, default 4), collects new tuples, and stops when no new data is returned.
 - **Duplicate Removal and JSON Recovery:** The system hashes tuple contents (via `normalise_row()`) to track and deduplicate results across iterations. It also attempts to recover valid JSON even from partially truncated or malformed LLM output.

Unlike Key-Scan, which explicitly extracts primary keys first and then populates attributes one by one, Table-Scan retrieves complete tuples directly using LLM in multiple iterations.

4.3 Table vs Key-Scan in GaloisWO

5 Logical Optimization (Query PLanning)

The Logical Optimization phase determines what information needs to be extracted from the LLM and which filtering conditions should be "pushed down" to the generative model versus applied locally. This decision-making process is critical to balancing the trade-off between retrieval precision and the risk of hallucination.

5.1 Predicate Push-Down Strategy

The system implements a flexible planning mechanism capable of generating different execution plans based on the configured optimization level. The planner analyzes the **WHERE** clause of the parsed SQL query and partitions the conditions into two sets: **conditions_to_push** (integrated into the prompt) and **residual_conditions** (deferred to the post-processor). The framework supports four distinct logical variants:

- **No-Push (GALOIS-WO)**: Treats the LLM purely as a data source. No filtering conditions are included in the prompt; the system retrieves the entire extension of the table (or its keys) and performs all filtering locally.
- **Push-All (GALOIS-F)**: Maximizes delegation by translating all SQL predicates into natural language constraints within the prompt. This assumes high model capability in understanding and applying complex logic.
- **Push-Selective (GALOIS-S)**: A heuristic approach that identifies and pushes only the most selective condition, using the LLM’s confidence self-assessment as a proxy to determine which conditions are “safe” or “selective” enough to be handled directly by the generative model and aims to reduce the search space without overwhelming the model context.
- **Push-Confident (GALOIS-F)**: An adaptive strategy that utilizes a **ConfidenceEstimator**. The system queries the LLM to self-evaluate its confidence in correctly assessing specific predicates (returning a "HIGH" or "LOW" score). Only conditions with high confidence scores are pushed down, while ambiguous ones are retained for local execution.

5.2 Join Handling

For multi-table queries, the logical planner decomposes the join operation into independent scan tasks. The system identifies all tables involved in the query (FROM clause and JOIN clauses) and generates a sub-plan for each. The planner ensures that alias references in the SQL query are correctly mapped to their respective table scans, allowing the Executor to fetch data for each relation independently before they are rejoined in the Post-Processing phase.

6 Physical Optimization (Execution Strategies)

While Logical Optimization defines the scope of the query, Physical Optimization defines the algorithmic strategy for data retrieval. Our system implements iterative algorithms designed to overcome the context window limitations of LLMs and to ensure comprehensive data extraction.

6.1 Table-Scan Operator

!!!!TO DO!!!!

6.2 Key-Scan Operator

!!!!TO DO!!!!

6.3 Adaptive Strategy Selection

The framework includes an **auto** execution mode that dynamically selects between Table-Scan and Key-Scan. This decision is mediated by the **ConfidenceEstimator**, which evaluates the query complexity and the number of requested columns. The confidence estimation logic incorporates a penalty factor based on the query’s complexity. The final confidence score C_{final} is not merely the raw probability returned by the model (C_{raw}), but is exponentially decayed by the number of selected columns (N_{cols}):

$$C_{final} = (C_{raw})^{N_{cols}}$$

If C_{final} is below a specific threshold (τ) or if the schema is particularly wide, the system defaults to the Key-Scan strategy to ensure data integrity, otherwise, it proceeds with the more direct Table-Scan.

More deeply this formulation models the increasing difficulty of retrieving coherent tuples as the schema width grows. Consequently, queries requesting a large number of attributes result in lower confidence scores, biasing the physical optimizer towards the safer Key-Scan strategy rather than a direct Table-Scan. This handling mode is useful for GALOIS-F logical plan.

7 Execution Cycle & Memory Handling

The interaction between the deterministic orchestration layer and the generative model is governed by a stateful execution cycle. Unlike standard single-turn LLM interactions, the Galois framework maintains a persistence layer across iterations to maximize recall and prevent redundancy. This logic is encapsulated within the **GaloisExecutor** class and its specialized internal memory component, **GaloisMemory**.

7.1 Stateful Context Management

!!!!EXPLAIN OF THE GALOISMEMORY OBJECT AND ITS FUNCTIONS !!!!!

7.2 The Iterative execution loop

!!!!EXPLAIN THE LOOP FOR RETRIEVAL PROCESS (MAX_ITERATIONS ECC...)!!!!

7.3 Termination Logic and Convergence

!!!!EXPLAIN UNDER WHICH CONDITION THE LOOP TERMINATES !!!!

8 Post-Processing

The Post-Processing module serves as the deterministic convergence point of the architecture, addressing the inherent impedance mismatch between the probabilistic, unstructured output of the Large Language Model and the strict typing requirements of

the SQL standard. This phase transforms the raw tuple sets retrieved by the Executor into a consistent relational state, enabling the execution of complex logical operations that cannot be reliably offloaded to the generative model.

8.1 Data Aggregation and Virtualization

Upon the completion of the physical scan phase (whether Table-Scan or Key-Scan), the retrieved tuples are aggregated into a local staging structure, referred to as the "**Data Lake**" which maps table names to lists of dictionaries. To bridge the gap to a relational environment, the system instantiates an ephemeral, in-memory DuckDB connection.

The aggregated data is converted into Pandas DataFrames and registered as virtual views within this database instance, effectively creating a transient relational schema populated by the LLM-generated data.

8.2 Heuristic Type Inference and Sanitization

The system implements a strict type inference policy to preserve data integrity. The `perform_local_join_and_query` method attempts to cast string-based LLM outputs into numeric types using a conservative heuristic: if the conversion to a numeric format introduces any new missing values (NaNs) that were not present in the original dataset—indicating a potential loss of information due to parsing failures—the entire column reverts to its original object (string) type. This zero-tolerance approach ensures that alphanumeric identifiers or mixed-format codes are not erroneously coerced into null values.

8.3 Deterministic Query Execution

Once the data is virtualized and typed, the system executes the logical query plan locally. This step involves running the original SQL query (sanitized of specific schema prefixes like `.target`) against the virtual tables. This local execution performs two vital functions:

- **Join Reconstruction:** It re-establishes the relational links between disparate datasets that were retrieved via independent scan operations, ensuring the correct implementation of INNER or LEFT joins defined in the original AST.
- **Residual Filtering:** It applies the `residual_conditions` predicates identified during the Logical Optimization phase as too complex or numerically precise for the LLM (e.g., strict inequalities like `length_in_km > 750`). This ensures that the final result set adheres strictly to the logical constraints of the user’s query, filtering out any hallucinations or imprecise data artifacts introduced by the model.

9 Numerical Results

In this section we will show a comparison between the metrics of original GALOIS paper written by the authors, and the metrics that we obtained from our GALOIS implementation.

9.1 GALOIS WO Results comparison

Metric	PAPER	Ours	$\Delta(\text{PAPER} - \text{Ours})$
F1-Cell	0.518	0.088	0.430
Cardinality	0.691	0.265	0.426
Tuple Constr.	0.389	0.048	0.341
AVG-Score	0.531	0.134	0.397
#Tokens (M)	19.710		
Avg Time	1460.0		

Table 4: GaloisWO comparison

9.2 GALOIS S Results comparison

Metric	PAPER	Ours	$\Delta(\text{PAPER} - \text{Ours})$
F1-Cell	0.480	0.434	0.046
Cardinality	0.655	0.667	-0.012
Tuple Constr.	0.365	0.297	0.068
AVG-Score	0.500	0.466	0.034
#Tokens (M)	0.960		
Avg Time	130.00		

Table 5: GaloisS comparison

9.3 GALOIS A Results comparison

Metric	PAPER	Ours	$\Delta(\text{PAPER} - \text{Ours})$
F1-Cell	0.543	0.417	0.126
Cardinality	0.799	0.614	0.185
Tuple Constr.	0.448	0.279	0.169
AVG-Score	0.592	0.436	0.156
#Tokens (M)	0.950		
Avg Time	120.50		

Table 6: GaloisA comparison

9.4 GALOIS F Results comparison

Metric	PAPER	Ours	$\Delta(\text{PAPER} - \text{Ours})$
F1-Cell	0.563	0.319	0.244
Cardinality	0.835	0.528	0.307
Tuple Constr.	0.464	0.166	0.298
AVG-Score	0.622	0.338	0.284
#Tokens (M)	1.720		
Avg Time	47.400		

Table 7: GaloisF comparison

10 Implementative challenges encountered

The most challenging aspect of the implementation was the development and the results enhancing for the **Key Scan** physical operator, in particular we obtained very poor performance in **GALOIS F** and **GALOIS W0** logical plans. This is mainly due to the fact in the concept of Key Scan we first ask to the LLM to retrieve data using only the primary key for each table, so for instance if we have the **venezuela_presidents** table with **president_name** defined as table's Primary Key, in the first phase we will ask to the LLM to retrieve the presidents of all word and only in the second phase we will apply the original query to the result set returned by the LLM; but in principle if the LLM didn't return any president from Venezuela the final result set post query filtering will be empty. This behavior is common to all datasets used in the **GALOIS F** and **GALOIS W0** versions, so we had to find a way to improve the Key Scan's retrieval process effectiveness.

Another issue we faced was managing DuckDB connections during execution, since several classes open connections to retrieve column data types or, more broadly, the database schema.

For address this aspect, the resource management for the DuckDB backend follows a strict acquisition-release pattern. Database connections are instantiated solely within the scope of the active schema retrieval or query execution task and are immediately released within finally blocks upon completion. This explicit lifecycle management prevents file locks on the underlying .duckdb instance, ensuring that the transition between the schema inspection phase and the subsequent in-memory virtualization phase proceeds without contention.

!!!!NOMINARE IL FATTO DI POTER OTTIMIZZARE LA STRUTTURA E LA SEMANTICA DEI PROMPT, VISTO CHE QUELLI USATO DA GALOIS NON SONO EFFICACI !!!!

11 Contributions

Alessio Demo (Badge number: 2142885)

•

Francesco Pivotto (Badge number: 2158296)

•

Giorgia Amato (Badge number: 2159999)

•